Capítulo 1 Rational Unified Process

Un proceso define "quién" está haciendo "qué", "cuándo" y "cómo" par alcanzar un determinado objetivo[1]. En este capítulo se defienen las principales características del Proceso Unificado de Desarrollo propuesto por Ivar Jacobson, james Rumbaugh y Grady Booch.

1.1- Proceso de Desarrollo de Software.

Un Proceso de Desarrollo de Software es la definición del conjunto de actividades que guían los esfuerzos de las personas implicadas en el proyecto, a modo de plantilla que explica los pasos necesarios para terminar el proyecto[2]. En la figura 1 se indica que este conjunto de actividades, en el proceso de desarrollo de software que proponen Jacobson, Rumbaugh y Booch, tiene la misión de transformar los requerimientos del usuario en un producto de software; de manera que los integrantes del equipo y todo aquel que pueda estar interesado en el producto final, tenga la misma visión y no ocurra cuando no se aplica un proceso de desarrollo (figura 2).

Figura 1 Proceso de Desarrollo de Software.

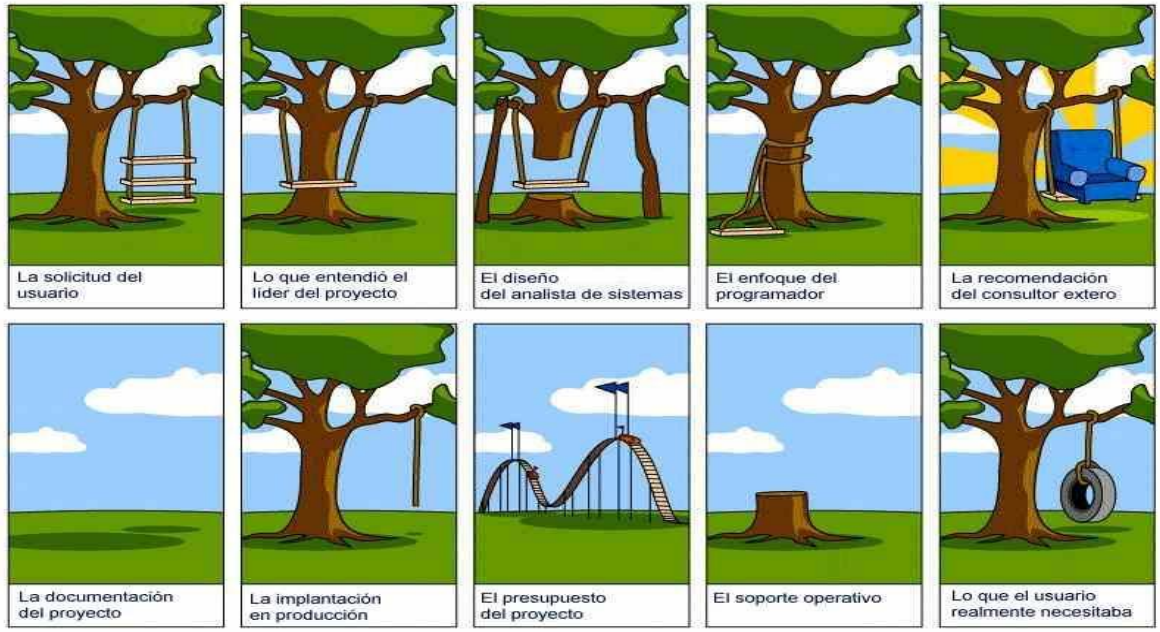


Figura 2 Visiones del producto final.

[1] Jacobson, I.; Booch, G. y Rumbaugh, J.; "El Proceso Unificado de Desarrollo de software". 2000. Página XVI.
[2] Jacobson, I.; Booch, G. y Rumbaugh, J.; "El Proceso Unificado de Desarrollo de software". 2000. Página 13.

Por lo tanto, las piedras angulares del proceso de desarrollo del software son: el poyecto, las personas y el producto; siendo las características del cliente, el entorno de desarrollo y las condiciones del negocio, elementos que influyen en el proceso (figura 3).

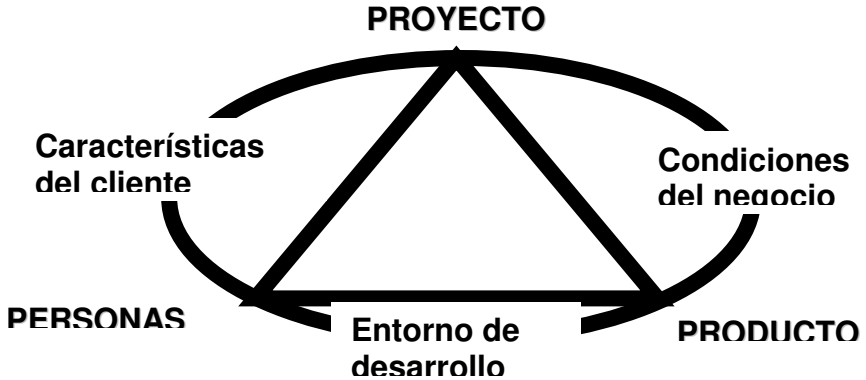


Figura 3 Piedras angulares del Proceso de Desarrollo de Software.

El conocimiento y la experiencia que crea y sostiene la evolución del producto lo tienen las personas. Ellas también financian, se benefician, lo prueban y planifican. Sin un personal competente y experimentado es imposible crear prosuctos competitivos que satisfagan las necesidades de los clientes. Además, el modo en que organiza y gestiona un producto (factibilidad del proyecto, definición del equipo que lo desarrolló, identificación y análisis de riesgos que implican su realización, planificación, nivel de entendimiento de lo que se está haciendo y sensación de que avanza en el cumplimiento) afectan a las personas involucradas en su realización.

Un proyecto es un elemento organizativo a través del cual se gestiona el desarrollo del software.

Un proyecto de desarrollo obtiene un versión de un producto que contiene modelos, código fuente, documentación y un ejecutable. Este producto va evolucionando durante el proceso de desarrollo desde un proyecto inicial o innovador (prototipo inicial) hasta convertirse en un release del proyecto.

En la figura 4 se aprecia como a cada nivel (organización, equipo, individuo) se han ido desarrollando métodos que mejoran el proceso en dependecia de las actividades que se realizan y las personas que participan. En este curso se estudiará un método desarrollado para su palicación por parte de un equipo de proyecto, en el que se dejan claros los roles que juegan los trabajadores dentro de los flujos de trabajo que los integran.

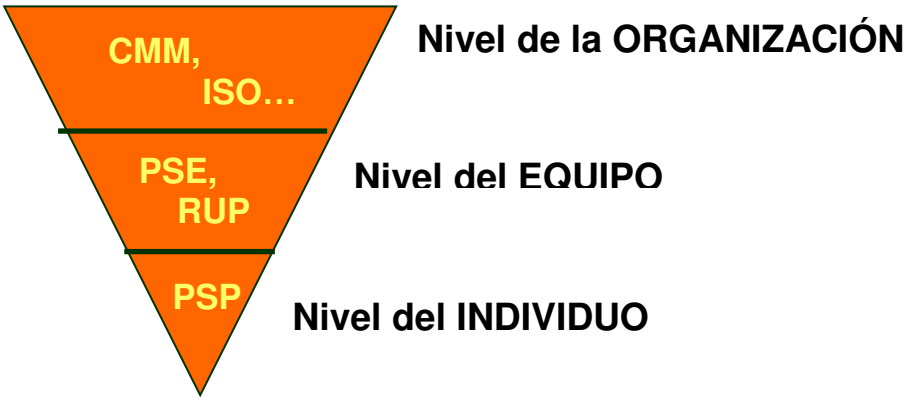


Figura 4 Métodos de mejora del proceso de desarrollo de software en los distintos niveles de implementación.

- CMM (Modelo de Maduración de Capacidades): Modelo que brinda un procedimiento para pasar de un proceso inmaduro que pocos conocen, no se mide, se aplica solo en ocasiones, es percibido como poco eficiente y es interpretado de manera distinta; a un proceso maduro definido, documentado, en el que el personal está entrenado en el proceso, practicdo, apoyado, mantenido, controlado, verificado, medido y meorable.
- ISO (Organización de Estándares Internacional): Define estpandares de calidad, las normas ISO 9001, ISO 9002, ISO 9003 e ISO 9004 contiene elementos asociado con el software.
- PSE (Proceso de software en Equipo): Ayuda a seguir de manera consistente los métodos disciplinados de ingeniería creado una disciplina en la que directivos (creen un ambiente donde el trabajo dsisciplinado sea notorio y reconocido), miembros (conozcan y estén convencidos del trabajo disciplinado) y el equipo (usen métodos de trabajo en equipo para planificar y establecer metas) trabajan en conjunto.
- RUP (Proceso Unificado de Rational - Proceso Unificado de Desarrollo de Software): Es un proceso que de manera ordanada defina las tareas y quién de los miembros del equipo de desarrollo las hará. Es una guía para usar UML.
- PSP (Proceso de Software Personal): Proceso que busca cambiar el compoprtamiento individual haciendo que las personas comprendan la necesidad de la aplicación de métodos disciplinados y separa organizar y planificar su trabajo, chequear su ejecución y dirigir la calidad del software.

1.2 - El Lenguaje Unificado de Modelación

UML es un lenguaje para visualizar, especificar, construir y documentar los artefactos de un sistema que involucra una gran cantidad de software[3] (figura 5).

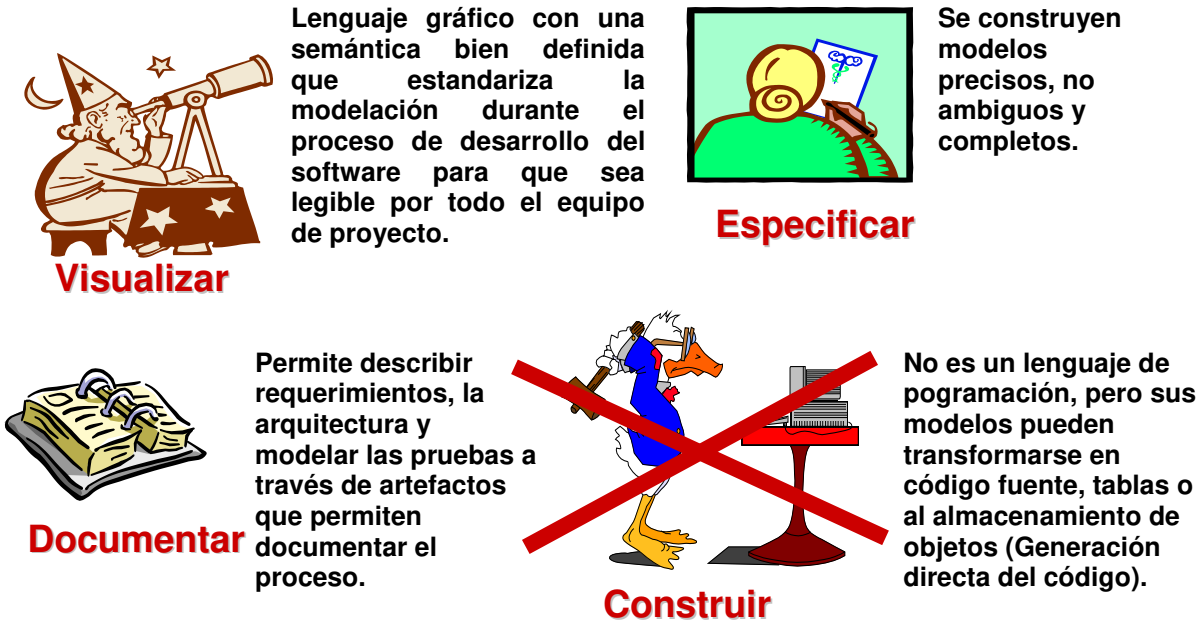


Figura 5 Definición de UML.

[3] Booch, G.: Rumbaugh, J. y Jacobson, I.; "El Lenguaje Unificado de Modelado". 2000. Página 11.

UML es el rsultado, en principo, de la unión de los métodos de Booch (Object Oriented Analysis and Desig with Application) y Rumabugh (OMT- Object Modeling Tecnique) para producir lo que en principo se conoció como el Método Unificado, pero que con la unión de Jacobson (OOSE-Object Oriented software Engineering: A use case driven approach) dio paso al Lenguaje Unificado de Modelación. En Noviembre de 1997 este lenguaje (en su versión 1.1) fue adoptado como el estándar por el OMG (Object Modeling Group).

Es importante recalcar que UML no es una guía para realizar el análisi y diseño orientado a objetos, es ecir, no es un proceso. UML es un lenguaje que pemite la modelación de sistemas con tecnología orientada a objetos.

UML no es el fruto de estos tres autores, con lo años y desde sus surgimiento se ha ennriquecido de diferentes trabajos (Figura 6).

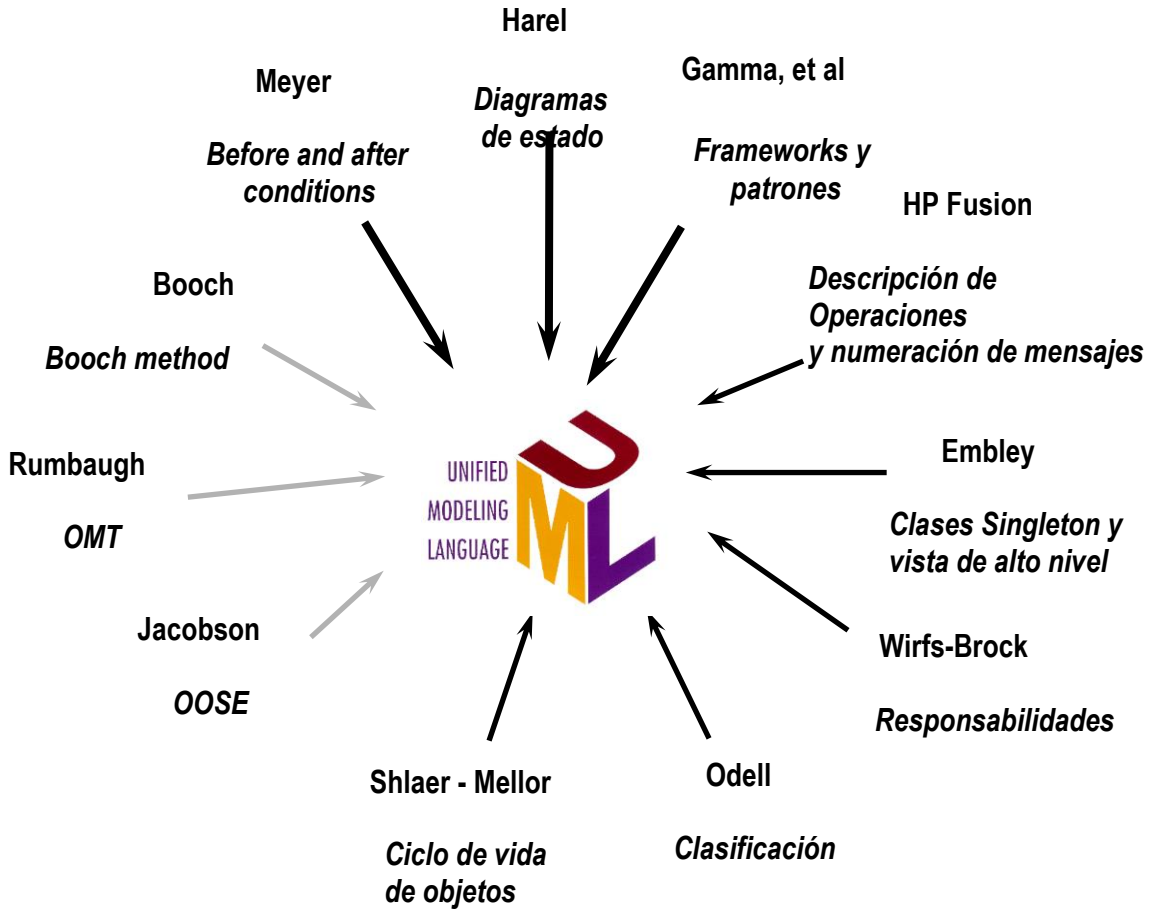


Figura 6 Contribuciones a UML.

El modelo gráfico de UML tiene un vocabulario en el que se identifican:

1. Elementos (abstracciones que constituyen los bloques básicos de construcción).
   - Estructurales: Partes que representan cosas.
     - Clase: Conjunto de oobjetos que comparten atributos, opraciones, relaciones y semántica.
     - Colaboración: Colección de operaciones que especifican un servicio de una clase o un componente.

    - Colaboración: Define la interacción entre los elementos que proporcionan un comportamiento cooperativo mayor que la suma de los comportamientos de sus elementos.
    - Caso de uso: Conjunto de secuencia de acciones que un sistema ejecuta y que produce un resultado observable para un actor.
    - Clase activa: Clase cuyos objetos tienen uno o más procesos o hilos de ejecución.
    - Componente: Es una parte física y reemplazable de un sistema que conforman un conjunto de interfaces y proporciona la implementación de dicho conjunto.
    - Nodo: Elemento físico que dispone de memoria y con frecuencia capacidad de almacenamiento.
  - Comportamiento: Partes del modelo que representan el comportamiento en el tiempo y el espacio.
    - Interacción: Conjunto de mensajes intercambiados entre un conjunto de objetos para alcanzar un propósito específico.
    - Máquina de estado: Esoecifica las secuencias de estados por las que pasa un objeto o una interacción durante su vida.
  - Agrupamiento: Cajas en las cuales puede descomponerse un modelo.
    - Paquete: Mecanismo de propósito general para organizar elementos en grupos.
  - Anotación: Comentarios que se pueden aplicar para describir, claroficar y hacer observaciones sobre cualquier elemento de un modelo.
2. Relaciones: Ligan los elementos.
  - Dependencia: relación semántica que indica que un cambio en un elemento afecta a la semántica de otro elemento.
  - Asociación: Relación estructural que describe las conexiones entre objetos.
  - Generalización/Especialización: Relación en la que el hijo comparte la estructura y el compotamiento del padre.
  - Realización: Relación semántica entre clasificadores, en donde un clasificador especifica un contrato que otro clasificador garantiza que cumplirá.
3. Diagramas: Es la representación gráfica de un conjunto de elemntos. Visualizan un sistema desde diferentes perspectivas.
  - Diagramas de estructura estática:
    - Diagrama de clases: Conjunto de clases, interfaces y colaboraciones; así como sus colaboraciones.
    - Diagrama de objetos: Conjunto de objetos y sus relaciones.
  - Diagramas de comportamiento:
    - Diagramas de interacción (secuencia y colaboración): Objetos y sus relaciones, incluyendo los mensajes que pueden ser enviados entre ellos.
    - Diagrama de estados: Muestra un máquina de estado que consta de estados, transiciones, eventos y actividades.

  - Diagrama de actividad: Es un tipo especial de diagrama de estados que muestra el flujo de actividades dentro de un sistema.
- Diagrama de casos de uso: Conjunto de casos de uso y actores y sus relaciones.
- Diagramas de implemenrtación:
  - Diagrama de componentes: Organización y las dependencias entre un conjunto de componentes.
  - Diagrama de despliegue: Configuración de nodos de procesamiento en tiempo de ejecución y los componentes que residen en ellos.

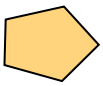
1.3 - El Proceso Unificado de Desarrollo de Software

RUP es el resultado de varios años de desarrollo y uso práctico en el que se han unificado técnicas de desarrollo, a través del UML, y trabajo de muchas metodologías utilizadas por los clientes. La versión que se ha estandarizado vió la luz en 1998 y se conoció en sus inicios como Proceso Unificado de Rational 5.0; de ahí las siglas con las que se identifica a este proceso de desarrollo.

Como RUP es un poceso, en su modelación define como sus principales elementos:

Trabajadores (“quién”)

Define el comportamiento y responsabilidades (rol) de un individuo, grupo de individuos, sistema automatizado o máquina, que trabajan en conjunto como un equipo. Ellos realizan las actividades y son propietarios de elementos.

Actividades (“cómo”) 

Es una tarea que tiene un propósito claro, es realizada por un trabajador y manipula elementos.

Artefactos (”qué”)

Productos tangibles del proyecto que son producidos, modificados y usados por las actividades. Pueden ser modelos, elementos dentro del modelo, código fuente y ejecutables.

Flujo de actividades (“Cuándo”)

Secuencia de actividades realizadas por trabajadores y que produce un resultado de valor observable.

En RUP se han agrupado las actividades en grupos lógicos definiéndose 9 flujos de trabajo princiaples. Los 6 primeros son conocidos como flujos de ingeniería y los tres últimos como de apoyo. En la figura 7 se representa el rpoceso en el que se grafican los flujos de trabajo y las fases y muestra la dinámica expresada en iteraciones y puntos de control.

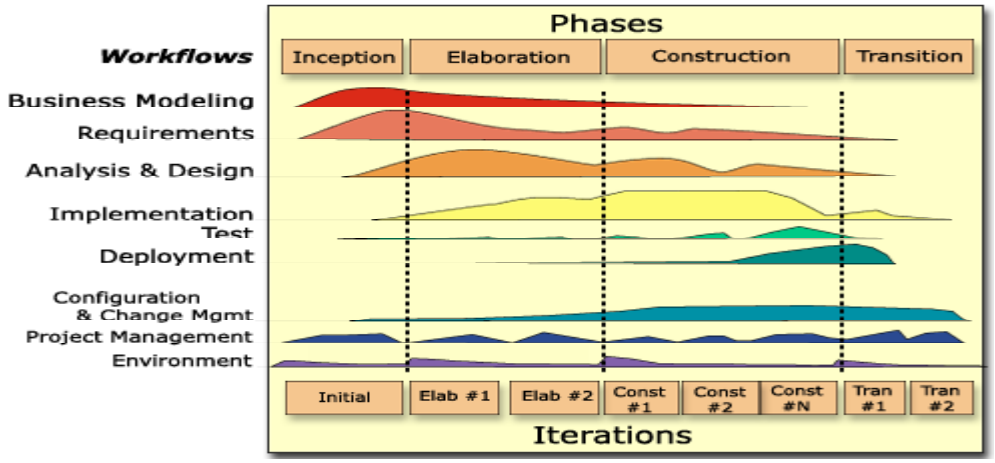


Figura 7 Fases-Flujos de trabajo.

Flujos de trabajo:

- **Modelamiento del negocio:** Describe los procesos de negocio, identificando quiénes participan y las actividades que requieren automatización.
- **Requerimientos:** Define qué es lo que el sistema debe hacer, para lo cual se identifican las funcionalidades requeridas y las restricciones que se imponen.
- **Análisis y diseño:** Describe cómo el sistema será realizado a partir de la funcionalidad prevista y las restricciones impuestas (requerimientos), por loq ue indica con precisión lo que se debe programar.
- **Implementación:** Define cómo se organizan las clases y objetos en componentes, cuáles nodos se utilizarán y la ubicación en ellos de los componentes y la estructura de capas de la aplicación.
- **Prueba (Testeo):** Busca los defectos a los largo del ciclo de vida.
- **Instalación:** Produce release del producto y realiza actividades (empaque, instalción, asistencia a usuarios, etc.) para entregar el software a los usuarios finales.
- **Administración del proyecto:** Involucra actividades con las que se busca producir un producto que satisfaga las necesidades de los clientes.
- **Administración de configuración y cambios:** Describe cómo controlar los elementos producidod por todos los integrantes del equipo de proyecto en cuanto a: utilización/actualización concurrente de elementos, control de versiones, etc.
- **Ambiente:** Contiene actividades que describen los procesos y herramientas que soportarám el equipo de trabajo del rpoyecto; así como el pocedimiento para implementar el proceso en una organización.

Fases:

- **Conceptualización (Concepción o Inicio):** Se describe el negocio y se delimita el proyecto describiendo sus alcances con la identificación de los casos de uso del sistema.
- **Elaboración:** Se define la arquitectura del sistema y se obtiene una aplicación ejecutable que responde a los casos de uso que la comprometen. A pesar de que se desarrolla a profundidad una parte del sistema, las decisiones sobre la arquitectura se hacen sobre la

base de la comprensión del sistema completo y los requerimientos (funcionales y no funcionales) identificados de acuerdo al alcance definido.

- **Construcción:** Se obtiene un producto listo pata su utilización que está documentado y tiene un manual de usuario . Se obtiene 1 o varios release del producto que han pasado las pruebas. Se ponen estos release a consideración de un subconjunto de usuarios.
- **Transición:** El release ya está listo para su instalación en las condiciones reales. Puede implicar reparación de errores.

El ciclo de vida de RUP se caracteriza por:

1. **Dirigido por casos de uso:** Los casos de uso reflejan lo que los usuarios futuros necesitan y desean, lo cual se capta cuando se modela el negocio y se representa a través de los requerimientos. A partir de aquí los casos de uso guían el proceso de desarrollo ya que los modelos que se obtienen, como resultado de los diferentes flujos de trabajo, representan la realziación de los casos de uso (cómo se llevan a cabo).
2. **Centrado en la arquitectura:** La arquitectura muestra la visión com+un del sistema completo en la que el equipo de proyecto y los usarios deben estar de acuerdo, por lo que describe los elementos del modelo que son más importantes para su construcción, los cimientos del sistema que son necesarios como base para comprenderlo, desarrollarlo y producirlo económicamente. RUP se desarrolla mediante iteraciones, comenzando por los CU relevantes desde el punto de vista de la arquitectura. Tal como se aprecia en la figura 8, el modelo de arquitectura se representa a travpés de en las que se incluyen los diagramas de UML.

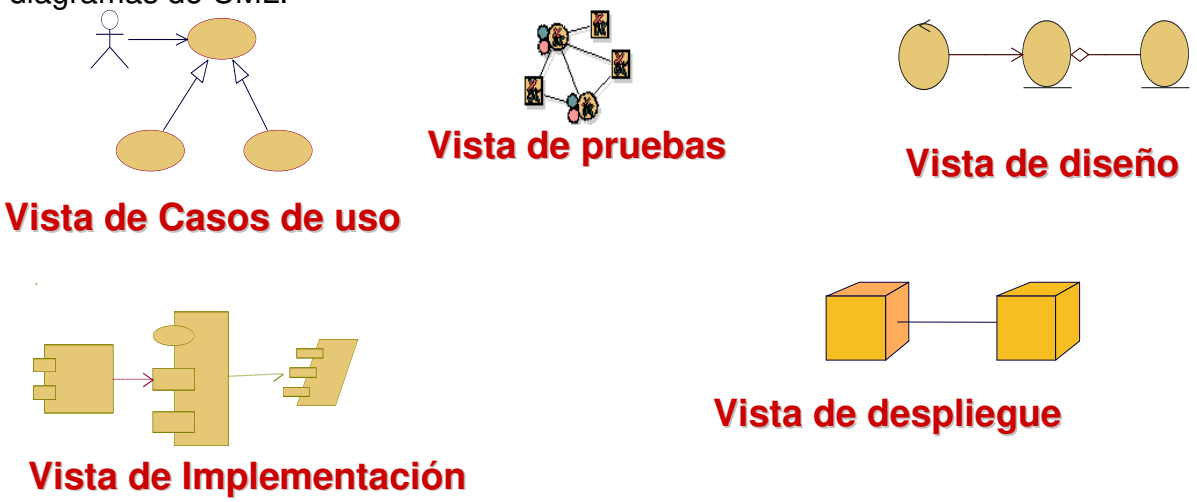


Figura 8 Vista del modelo de arquitectura.

3. **Iterativo e Incremental:** Aunque la figura 7 puede sugerir que los flujos de trabajon se desarrollan en cascada, la “lectura” de este grpafico tiene que ser vertical y horizontal. RUP propone que cada fase se desarrolle en iteraciones. Una iteración involucra actividades de todos los flujos de trabajo, aunque desarrolla fundamentalmente algunos más que otros. Por ejemplo,u na iteración de elaboración centra su atención en el análisis y diseño, aunque refina los requerimientos y obtiene un producto con un determinado nivel, pero que irá creciendo incrementalmente en cada iteración.

   Aunque cada iteración tiene que proponerse un incremento en el proceso de desarrollo, todas deben aportar al principal resultado de la fase en la que se desarrolla, por lo que los puntos de control evalúan:

- Conceptualización Objetivos
- Elaboración Arquitectura
- Construcción Funcionalidad operativa
- Transición Release del sistema

Para lograr estos cuatro hitos, hay que construir determinados artefactos definidos dentro de los flujos de tarbajo involucrados.

1.4 - Productos de Rational

La suite de Rational ofrece varios productos, destacándose los siguientes:

- **Rational Requisite Pro** – Mantiene a todo el equipo de desarrollo actualizado a través del proceso de desarrollo de aplicaciones haciendo que los requerimientos se puedan escribir, comunicar y cambiar fácilmente.
- **Rational ClearQuest** - Un producto Windows y basado en Web de administración de solicitudes de cambio que permite a los equipos de proyecto rastrear y administrar todas las actividades de cambio que ocurren durante el desarrollo del ciclo de vida.
- **Rational Rose** - La herramienta líder en el mundo de modelización visual para el proceso de modelización del negocio, análisis de requerimientos y diseño de arquitectura de componentes.
- **Rational SoDA** – Automatiza la producción de documentación para todo el proceso de desarrollo de software, reduciendo dramáticamente el tiempo y el costo de documentar el software.
- **Rational ClearCase** – Herramienta de administración de configuración de software, líder en el mercado, que da a los administradores de proyecto la posibilidad de rastrear la evolución de cada proyecto de desarrollo de software.

1.5 - Conclusiones

RUP brinda un proceso integrado que utiliza el estándar de notación UML para pemitir desarrollar un proceso de forma iterativa e incremental a partri de la identificación e implementación de los casos de uso.

1.6 - Referencias Bibliografías

- Jacobson, I.; Booch, G. y Rumbaugh, J.; “El Proceso Unificado de Desarrollo de software”. 2000. Addison-Wesley. Prólogo, Capítulos 1-5 y Apéndices A y B. Páginas 3-104, 407-424..
- Booch, G.: Rumbaugh, J. y Jacobson, I.; “El Lenguaje Unificado de Modelado”. 2000. Addison-Wesley.
- Pressman, Roger; Ingeniería de software. Un enfoque práctico. 2002. McGraw.Hill/Interamericana de España. Prólogo y Capítulos 1, 2, 3 y 11. Páginas 3-48 y 181-195.